

# Динамический полиморфизм в C++

Механика vtable, цена абстракции и современные альтернативы

Максим Емельянов    Литвинов Юрий

C++ Семинар

5 мая 2026 г.

Часть 1: Механика vtable

Часть 2: Классический подход и шаблоны

Часть 3: `std::variant`

Часть 4: Полиморфизм в Go

Часть 5: Полиморфизм в Rust

Часть 6: Сравнение подходов

# Что такое полиморфизм? Раннее vs Позднее связывание

- ▶ **Динамический полиморфизм** — механизм, при котором программа определяет точный адрес вызываемой функции во время выполнения программы (Runtime), исходя из реального типа объекта в памяти.
- ▶ **Раннее связывание (Early binding):**  
Вызов функции привязывается к указателю на этапе компиляции. Не подходит для восходящего преобразования (upcast).
- ▶ **Позднее связывание (Late binding):**  
Гарантирует вызов правильной версии функции для конкретного объекта, независимо от типа ссылки. В C++ реализуется через `virtual`.

# Взгляд под капот: vptr и vtable

При добавлении `virtual` компилятор неявно добавляет в объект указатель **vptr**, который ссылается на **vtable** (таблицу виртуальных методов).

## Как выглядит вызов в ассемблере (GCC):

```
mov rbp, QWORD PTR [rbx]    ; В регистр rbp помещаем указатель на объект
mov rax, QWORD PTR [rbp+0]   ; В rax помещаем указатель на vtable - разыменование 1
call [QWORD PTR [rax]]       ; Вызов функции по адресу из vtable - разыменование 2
```

- ▶ **Преимущество:** Правильный вызов метода дочернего класса.
- ▶ **Недостаток:** Затраты на двойное обращение к памяти.

# Классический пример: Иерархия (C++98)

```
class Property {  
protected:  
    double worth;  
public:  
    Property(double worth) : worth(worth) {}  
    virtual double getTax() const = 0; // Чисто виртуальный метод  
};  
  
class Car : public Property {  
public:  
    Car(double worth) : Property(worth) {}  
    double getTax() const override { return worth / 200; }  
};  
  
// Использование:  
Property* p = new Car(400000);  
std::cout << p->getTax(); // Вызовется Car::getTax()
```

# Статическая имитация (Шаблоны)

Можно избежать `vtable`, используя шаблоны:

```
template <typename T>
class Base {
public:
    void someFunctionality() {
        static_cast<T*>(this)->someFunctionality();
    }
};

class Child : public Base<Child> {
public:
    void someFunctionality() {
        cout << "Child class perform new function..." << endl;
    }
};
```

**Итог:** Ускоряет работу (нет `vtable`), но может сильно раздуть бинарник (каждый тип генерирует новый класс).

# Альтернатива C++17: std::variant + std::visit

```
class Derived {
public: void PrintName() const { std::cout << "calling Derived!\n"; }
};
class ExtraDerived {
public: void PrintName() const { std::cout << "calling ExtraDerived!\n"; }
};

int main() {
    std::variant<Derived, ExtraDerived> var = Derived{};

    // Вызов через лямбду (visitor)
    auto caller = [](const auto& obj) { obj.PrintName(); };
    std::visit(caller, var);
}
```

# Плюсы и минусы std::variant

## Преимущества:

- ▶ **Value semantics:** объекты хранятся без `new/delete`.
- ▶ **Нет базы:** классы могут быть не связаны иерархией.
- ▶ **Duck typing:** сигнатуры не обязаны строго совпадать.

## Недостатки:

- ▶ Все типы должны быть известны **заранее** (плохо для плагинов).
- ▶ **Перерасход памяти:** размер равен самому большому типу + тег (теряем байты на мелких объектах).



# Альтернативный взгляд: Полиморфизм в Go

- ▶ **Duck Typing:** неявная реализация интерфейсов.
- ▶ Нет наследования.
- ▶ Если тип содержит методы интерфейса — он его реализует.

```
type Geometry interface {  
    Area() float64  
}  
  
type Rect struct { Width, Height float64 }  
func (r Rect) Area() float64 { return r.Width * r.Height }  
  
func Measure(g Geometry) {  
    fmt.Println(g.Area())  
}  
  
func main() {  
    r := Rect{Width: 3, Height: 4}  
    Measure(r) // Работает автоматически  
}
```

# Взгляд на реализацию интерфейсов в Go

Переменная интерфейса в Go (`iface`) — это структура из двух указателей:

```
type iface struct {  
    tab *itab          // Указатель на таблицу интерфейсов  
    data unsafe.Pointer // Указатель на реальные данные  
}
```

- ▶ **itab (Interface Table)**: содержит метаданные типа и список указателей на функции. Схожа с `vtable` в C++.
- ▶ **Динамика и кэш**: `itab` генерируется в рантайме при первом сопоставлении пары "Интерфейс-Тип" и кэшируется.
- ▶ **Экономия**: в отличие от C++, объект не содержит `vptr`, если он не используется через интерфейс.

# Динамический полиморфизм в Rust: Traits

- ▶ **Треиты (Traits):** Описывают общее поведение типов.
- ▶ **dyn:** Ключевое слово для обозначения динамической диспетчеризации.
- ▶ **Box<dyn Trait>:** Так как размер объекта неизвестен при компиляции, работа всегда идет через указатель.

```
trait Animal { fn make_sound(&self); }
struct Dog;
impl Animal for Dog {
    fn make_sound(&self) { println!("Woof!"); }
}

fn main() {
    let animals: Vec<Box<dyn Animal>> = vec![
        Box::new(Dog),
        Box::new(Cat),
    ];
    for a in animals { a.make_sound(); }
}
```

# Под капотом Rust: Fat Pointers

В отличие от C++, Rust не хранит `vptr` внутри объекта. Вместо этого используется **толстый указатель** (16 байт на x64):

| Data Pointer (8 byte) | vtable Pointer (8 byte) |
|-----------------------|-------------------------|
| Адрес данных в памяти | Адрес таблицы методов   |

- ▶ **Преимущество:** Объекты «чистые», не занимают лишнюю память, если не используются полиморфно.
- ▶ **vtable содержит:**
  - ▶ Указатель на деструктор.
  - ▶ Размер типа и выравнивание (*size*, *align*).
  - ▶ Адреса реализаций методов трейта.

Хотя оба языка имеют похожие механизмы полиморфизма, имеются следующие различия:

## 1. Статика (Rust) vs Динамика (Go):

- ▶ В Rust `vtable` строится **на этапе компиляции**. Вызов максимально дешев.
- ▶ В Go `itab` может генерироваться **в рантайме** (через хеш-таблицы) при первом приведении типов.

## 2. Явность (Explicit) vs Неявность (Implicit):

- ▶ В Go реализация неявная: совпали сигнатуры — интерфейс реализован.
- ▶ В Rust требуется явный `impl Trait for Struct`, что исключает ошибки «случайного» совпадения имен.

# Итоговое сравнение подходов

| Критерий   | C++ Virtual      | C++ variant     | Go             | Rust             |
|------------|------------------|-----------------|----------------|------------------|
| Связывание | Runtime (vtable) | Runtime (index) | Runtime (itab) | Runtime (vtable) |
| Разметка   | Содержит vptr    | Данные + тег    | Чистый объект  | Чистый объект    |
| Расширение | Легко            | Сложно          | Легко          | Легко            |
| Инлайнинг  | Почти никогда    | Часто           | Редко          | Редко            |

# Заключение: Что быстрее?

- ▶ **std::variant** — потенциально самый быстрый за счет локальности данных и инлайнинга (для небольшого набора типов).
- ▶ **vtable (C++)** — стандарт для больших иерархий. Основные затраты: хранение `vptr` в каждом объекте и двойное разыменовывание при вызове.
- ▶ **Rust dyn** — предлагает баланс: объекты остаются «чистыми» (без `vptr`), а таблицы методов генерируются во время компиляции. Это дает производительность уровня C++.
- ▶ **Интерфейсы Go** — гибкий подход (Duck Typing). Удобен для крупных систем, но несет небольшие расходы на построение таблиц `itab` в рантайме.
- ▶ **Выбор зависит от задачи:**
  - ▶ Максимальная скорость: `std::variant` / Rust.
  - ▶ Гибкость архитектуры: Go / Rust.
  - ▶ Сложные иерархии наследования: C++ (`virtual`).